



Kafka 이론(1) - Producer 와 Consumer

☼ 상태	완료
≡ 다중 선택	

출처)

참고로 주황색으로 밑줄 친 부분은 출처를 그래도 가져온 블록임 (1-4. Kafka의 특징

1. Kafka 소개

대규모 분산 시스템에서 데이터를 실시간으로 수집, 전달, 처리할 수 있도록 설계된 이벤트 스트리밍 플랫폼

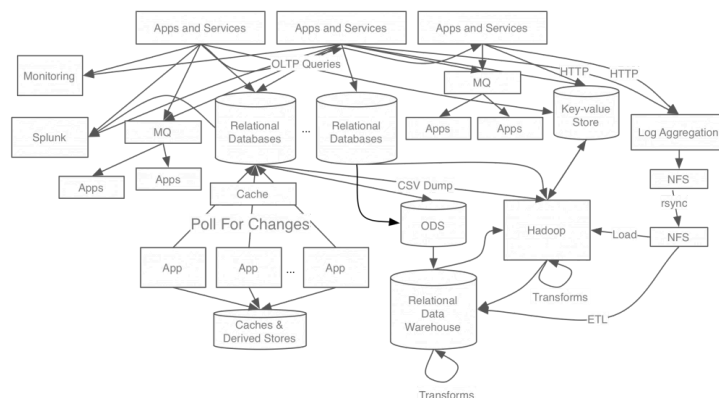
1-1. Kafka 탄생 이유

2010년에 LinkedIn에서 처음 개발되었고, 현재는 Apache 재단에서 오픈소스로 관리되고 있음

"LinkedIn에서는 어떤 문제를 해결하기 위해 Kafka를 만들었을까?"

Kafka 개발 전 : end-to-end 방식 (서비스 간 직접 연결)

Kafka가 등장하기 전에는 보통 시스템끼리 직접 데이터를 주고받는 방식을 많이 사용



예를 들어, 쇼핑몰 서비스에서 주문이 발생하면,

주문 서버가 결제 서버, 재고 서버, 배송 서버, 추천 서버, 분석 서버 등에 직접 데이터를 보내는 구조

• end-to-end 방식은 처음에는 단순해 보이지만, 시스템이 많아질수록 다음과 같은 문제가 커진다

▼ 중앙화된 데이터 전송 구조의 부재 → 시스템이 늘어날수록 복잡도 증가

- 중앙에서 데이터를 모아 전달해주는 구조가 없기 때문에, 새로운 시스템이 추가될 때마다 기존 시스템과의 연결을 새로 만들어야 한다.
- 각 시스템이 필요한 데이터를 직접 다른 시스템에서 받아와야 한다

▼ 시스템 간 강한 의존성 → 장애 발생 시 여러 시스템을 함께 확인해야 함

- 시스템들이 서로 직접 연결되어 있으면, 한 시스템의 문제가 다른 시스템에도 영향을 줄 수 있다.

• 기존의 Hadoop 기반 구조(배치 처리 중심)으로는 → 실시간 처리가 어렵다

▼ 기존의 Hadoop 기반 구조는 일정시간 대량의 데이터를 모아서 한 번에 처리하는 배치 처리에 강했음(대용량 데이터 분석에 적합)

- 예를 들어, 하루 동안 쌓인 로그를 밤에 한 번 처리하거나,
- 1시간마다 데이터를 모아서 분석하는 방식

▼ 배치 처리는 사용자의 행동에 즉시 반응해야 하는 서비스에는 한계가 있다

- 예를 들어, 사용자가 상품을 클릭했을 때 바로 추천 결과를 바꾸거나,
- 이상 거래를 즉시 탐지하거나,
- 실시간 대시보드에 바로 반영하는 작업

• 저지연 기반 서비스 구현의 한계

▼ 현대 서비스에서는 사용자의 행동 데이터를 빠르게 수집하고, 거의 실시간으로 처리하는 것이 중요

- 실시간 추천
- 실시간 로그 분석
- 실시간 이상 탐지
- 실시간 재고 반영
- 실시간 사용자 행동 추적

하지만 기존 End-to-End 방식이나 배치 중심 구조에서는 데이터를 빠르게 전달하고 처리하기 어렵다.

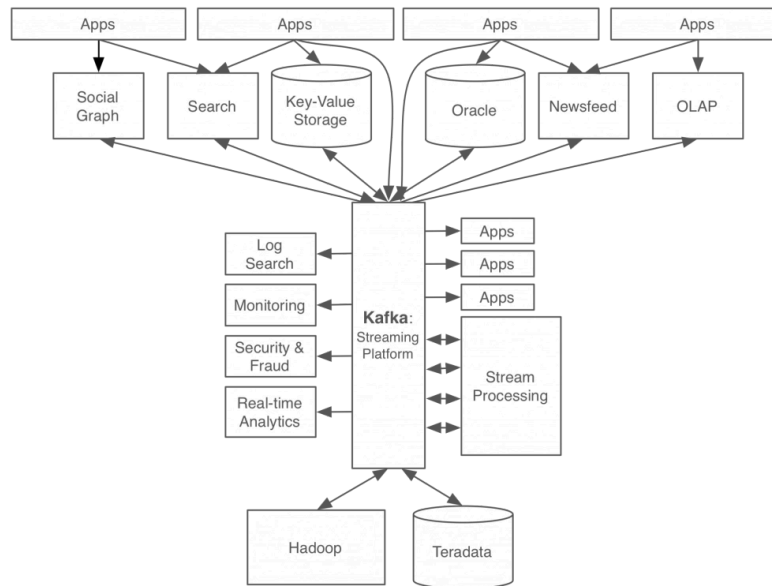
[정리]

데이터가 여러 시스템 사이를 직접 오가고(end-to-end), 처리도 일정 시간마다 몰아서 수행(배치 처리)되면 실시간성이 떨어진다. 결국 사용자의 행동이 서비스에 반영되기까지 시간이 오래 걸린다. (저지연 기반 서비스 구현의 한계)

- Kafka 이전의 구조는 시스템 수가 적을 때는 단순하게 사용할 수 있었지만, 시스템이 많아질수록 연결 관계가 복잡해지고 장애 추적이 어려워졌다.
또한 Hadoop 중심의 배치 처리 구조는 대용량 데이터 분석에는 적합했지만, 실시간 데이터 처리와 저지연 서비스 구현에는 한계가 있었다.
- 그래서 여러 시스템이 데이터를 직접 주고받는 방식이 아니라,
중앙의 데이터 스트리밍 플랫폼을 통해
① 데이터를 안정적으로 전달하고, ② 실시간 처리와 확장이 가능한 구조가 필요해졌다.

1-2. Kafka : 데이터 흐름을 중앙에서 관리하는 방식

Kafka 기반 구조는 데이터를 시스템끼리 직접 주고받는 방식이 아니라, Kafka를 중심에 두고 데이터 흐름을 관리하는 방식



- 즉, 각 시스템이 서로 직접 연결되는 것이 아니라 Kafka라는 공통 통로를 통해 데이터를 주고받는다
- 예를 들어 쇼핑몰에서 사용자가 상품을 클릭하거나 장바구니에 담거나 결제하면,
 - 해당 이벤트는 Kafka로 먼저 전달된다
 - 이후 추천 시스템, 로그 분석 시스템, 실시간 대시보드, 데이터 웨어하우스 등이 Kafka에서 필요한 데이터를 가져가 처리한다

① Kafka를 중심으로 데이터 흐름을 중앙에서 관리

Kafka는 여러 시스템 사이에서 데이터를 전달하는 **중앙 허브 역할**

- 기존 방식에서는 주문 서버가 결제 서버, 재고 서버, 분석 서버에 각각 직접 데이터를 보내야 했다.
- 하지만 Kafka 구조에서는 주문 서버가 Kafka에 이벤트를 한 번만 전달하면 된다.
 - 그 이후 필요한 시스템들이 Kafka에서 데이터를 가져가면 된다.

Kafka는 데이터가 이동하는 공통 통로 역할

따라서 각 시스템이 서로 직접 연결될 필요가 줄어들고, 데이터 흐름을 한 곳에서 관리하기 쉬워진다.

② 모든 시스템이 Kafka를 통해 데이터를 생산하고 소비

Kafka에서는 데이터를 보내는 시스템을 **Producer**, 데이터를 읽는 시스템을 **Consumer**라고 한다.

하나의 시스템이 Producer이면서 동시에 Consumer가 될 수도 있다.

각 시스템은 Kafka에 데이터를 보내거나, Kafka에서 필요한 데이터를 읽는 방식으로 동작

- 예를 들어 상품 클릭 이벤트를 발생시키는 웹 서버는 Producer
- 반대로 그 클릭 데이터를 읽어서 추천에 활용하는 추천 시스템은 Consumer

③ 스트림 처리 기반으로 실시간 데이터 처리 가능

Kafka는 데이터가 발생하는 즉시 이벤트를 전달할 수 있다.

(데이터를 일정 시간 동안 모았다가 한 번에 처리하는 배치 방식과 달리)

- 예를 들어 사용자의 클릭, 장바구니 추가, 결제 같은 행동 데이터를 거의 실시간으로 수집하고 처리할 수 있다.
 - 사용자가 특정 상품을 클릭하면

- 그 이벤트가 Kafka에 저장되고
- 추천 시스템이 바로 읽어서
- 다음 추천 결과에 반영할 수 있다

Kafka는 실시간 로그 분석, 실시간 추천, 이상 탐지, 모니터링 등에 적합하다

④ 시스템 간 의존성 감소 → 장애 발생 시 영향 범위 최소화

Kafka를 사용하면 시스템들이 서로 직접 연결되지 않아도 된다.

- 예를 들어 추천 시스템에 장애가 발생해도, 주문 서버나 결제 서버가 추천 시스템으로 직접 데이터를 보내고 있는 구조가 아니기 때문에 전체 서비스에 미치는 영향이 줄어든다.
 - 추천 시스템이 복구된 뒤에는 Kafka에 쌓여 있던 데이터를 다시 읽어 처리할 수도 있다.

Kafka가 중간에서 데이터를 저장하고 전달하기 때문에, 데이터를 생산하는 시스템과 소비하는 시스템이 느슨하게 연결된다. 이로 인해 한 시스템의 장애가 다른 시스템으로 바로 전파되는 것을 줄일 수 있다.

⑤ 새로운 시스템은 Kafka에 연결만 하면 되어 확장 용이

기존 구조에서는 새로운 시스템을 추가할 때마다 기존 시스템들과 직접 연결을 만들어야 했다.

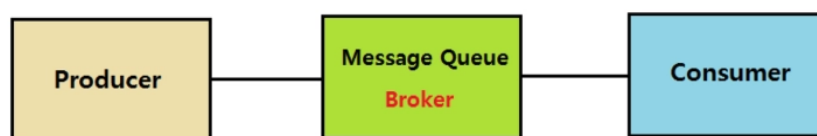
하지만 Kafka 구조에서는 새 Consumer 시스템이 Kafka의 특정 **“토픽”**을 읽도록 만들면 된다.

- 예를 들어 기존에 클릭 로그를 추천 시스템만 사용하고 있었다고 하자.
나중에 실시간 대시보드가 필요해지면, 대시보드 시스템(Consumer)도 Kafka의 클릭 로그 토픽을 구독하면 된다.

→ 기존 웹 서버나 추천 시스템의 구조를 크게 바꾸지 않아도 된다.

새로운 기능이나 분석 시스템을 추가할 때
Kafka에 저장된 데이터를 읽는 Consumer만 추가하면 되기 때문에 시스템 확장이 비교적 쉽다.

1-3. Kafka 구성과 동작 방식



MQ 동작 원리

- Kafka는 **Pub-Sub 모델** 기반으로 동작함
어떤 시스템이 데이터를 특정 공간에 발행하면, 그 데이터를 필요로 하는 다른 시스템들이 구독해서 가져가는 방식
 - 데이터를 보내는 쪽과 받는 쪽이 직접 연결되지 않고, 중간 매개체(Message Queue)를 통해 데이터를 주고받는 구조
 - Pub = Publish : 데이터를 발행한다는 의미 (Producer)
 - Sub = Subscribe : 필요한 데이터를 구독한다는 의미 (Consumer)
- Kafka를 이해하기 위해서는 메시지 큐와 메시지/이벤트 브로커 대한 이해가 필요함

메시지 큐 (Message Queue)

메시지 큐는 **Queue**라는 자료구조를 기반으로 메시지를 전달하는 시스템

→ 먼저 들어온 데이터가 먼저 처리됨 (FIFO 구조)

⇒ 시스템 간 의존성이 줄어들음 (시스템 간 직접 통신하지 않기 때문에)

- 메시지 큐는 **메시지 지향 미들웨어(Message Oriented Middleware)** 개념을 구현한 시스템

메시지 지향 미들웨어란, 서로 다른 시스템들이 직접 연결되지 않아도 메시지를 통해 데이터를 주고받을 수 있게 해주는 중간 소프트웨어

- 메시지 큐에서는 메시지를 보내는 쪽과 가져가는 쪽의 역할이 분리됨
 - **Producer** : 메시지를 생성하여 Queue에 전달하는 역할
 - **Consumer** : Queue에 저장된 메시지를 가져와 소비하는 역할
- **Producer는 메시지를 Queue에 전달하기만 하고, Consumer의 처리 결과를 기다릴 필요가 없음**
 - ⇒ 작업을 요청한 후 결과를 기다리지 않고 다음 작업을 수행할 수 있음
 - ⇒ **비동기 처리**가 가능해짐

메시지 브로커 / 이벤트 브로커 (서버)

메시지 큐 개념을 기반으로 구현된 시스템은 **데이터 처리 방식에 따라** 메시지 브로커와 이벤트 브로커로 나뉨

cf) Kafka는 여러 개의 브로커(서버)로 구성된 클러스터 형태로 동작



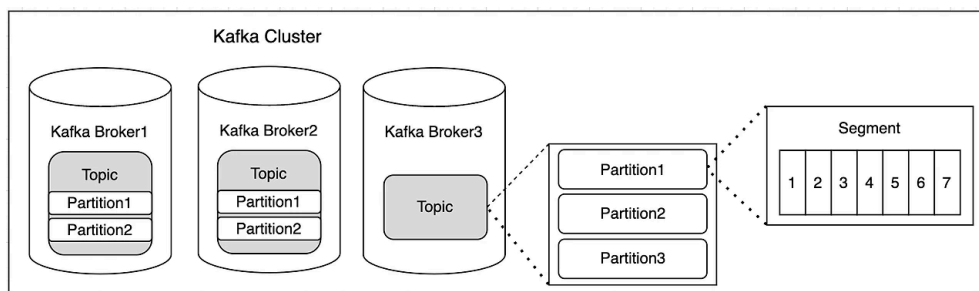
- **메시지 브로커(서버)**
 - Publisher가 생산한 메시지를 Queue에 저장하고, Consumer가 해당 메시지를 가져갈 수 있도록 중간 역할을 수행
 - 이처럼 **Publisher가 메시지를 발행하고, Consumer가 필요한 메시지를 구독하여 처리하는 구조를 Pub/Sub 모델**이라고 함
 - Pub/Sub 모델은 Publisher와 Consumer가 직접 연결되지 않아도 메시지를 주고받을 수 있게 하므로, 서로 다른 시스템 사이에서 데이터를 비동기적으로 전달하는 데 사용됨
 - 대부분의 경우 **Consumer가 데이터를 가져가면 즉시 또는 짧은 시간 내에 Queue에서 삭제되는 특징**이 있음
- **이벤트 브로커(서버)**
 - 메시지 브로커의 큐 기능을 포함하면서, publisher가 생산한 이벤트를 **이벤트 처리 후에 바로 삭제하지 않고 저장**
 - 이벤트 시점이 저장되어 있어서 consumer는 특정 시점부터 데이터를 다시 읽을 수 있음
 - 메시지 브로커 보다는 더 많은 양의 데이터를 처리할 수 있음

1-4. Kafka 특징

- **높은 처리량 (High Throughput)** : 많은 데이터를 빠르게 처리할 수 있는 특성
 - Kafka는 대용량 데이터를 실시간으로 처리할 수 있도록 설계됨
 - 데이터를 묶어서 압축 전송하여 전송 효율을 높임
 - 데이터를 여러 단위로 나누어 처리하여 많은 데이터를 동시에 처리 가능

- **확장성(Scalability)** : 상황에 따라 시스템을 쉽게 확장할 수 있는 특성
데이터 양은 상황에 따라 크게 변화하기에, 미리 정확한 트래픽을 예측하기 어려움
필요에 따라 서버를 추가하는 방식(수평 확장)으로 처리량을 늘릴 수 있음
- **영속성(Permanent Storage)** : 데이터가 사라지지 않고 유지되는 특성
Kafka의 데이터는 메모리가 아닌 디스크에 저장됨
데이터를 순서대로 계속 저장하기 때문에, 과거 데이터를 다시 읽고 재처리할 수 있음
- **고가용성(High Availability)** : 일부 장애가 발생해도 서비스가 계속 동작하는 특성
Kafka는 데이터가 한 곳에만 의존하지 않도록 분산 저장되는 구조
하나의 서버에 문제가 발생하더라도, 다른 서버에 있는 데이터를 통해 서비스가 계속 동작 가능

2. Kafka 기본 구조

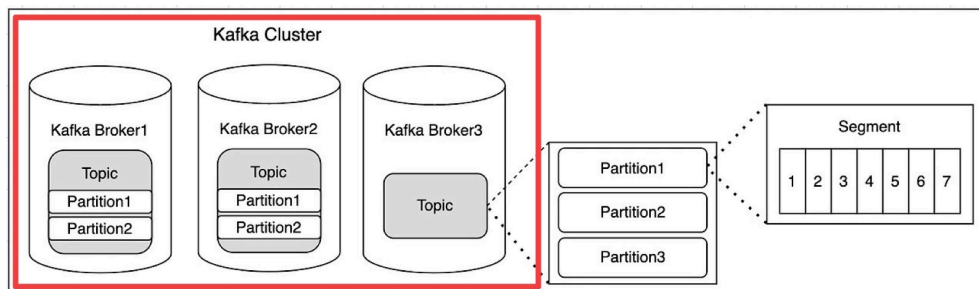


💡 데이터를 계층적으로 저장하는 구조!

```

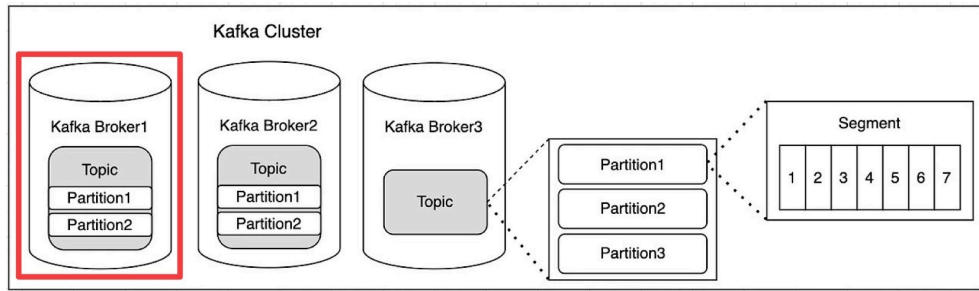
Cluster
├── Broker
│   ├── Topic
│   │   └── Partition
│   │       └── Segment
    
```

3-1. Cluster (클러스터)



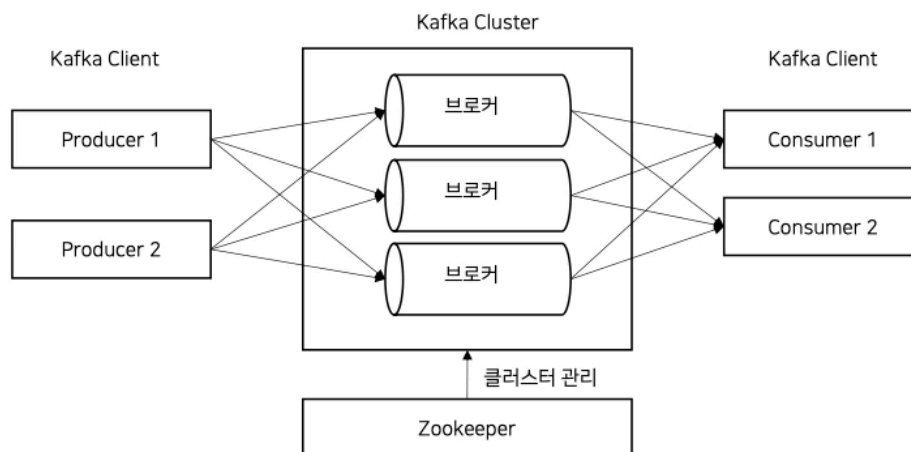
- Kafka는 하나의 서버가 아니라 여러 개의 서버(Broker)로 구성된 클러스터 형태로 동작
- 그림에서처럼 Kafka Broker 1, Broker 2, Broker 3은 각각 독립적인 서버(Broker)이며 함께 Kafka Cluster를 구성함

3-2. Broker (브로커,서버)



- 브로커는 실제로 데이터를 저장하는 역할을 수행
- 하나의 Broker에는 여러 토픽(Topic)과 파티션(Partition)이 존재하며, 데이터를 분산 저장함
- Broker 중 하나가 **Controller** 역할을 수행하고, 나머지 Broker들은 **데이터 처리 담당**
 - Controller : 클러스터의 상태 정보를 기반으로 브로커 상태를 관리하고, **리더 파티션(Leader Partition)**을 결정
 - 이때 클러스터의 상태 정보는 **Zookeeper**를 통해 공유됨
 - Controller에 장애가 발생하면, 다른 브로커가 해당 리더 역할을 이어받음
 - 시스템이 계속 동작하도록 & Controller는 항상 1개 존재

3-2-1. ZooKeeper

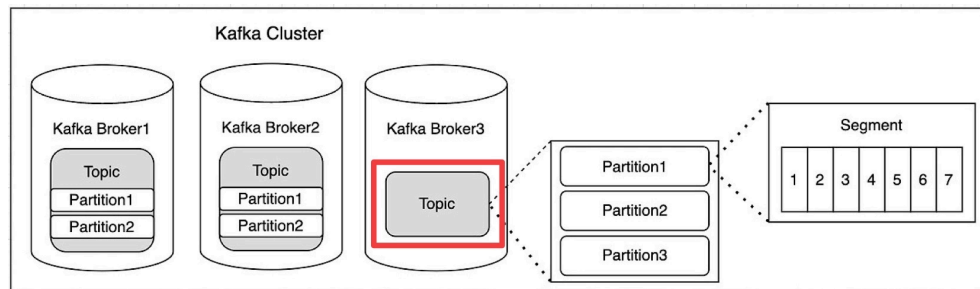


- 클러스터의 상태 정보 즉, 메타데이터(토픽, 파티션, 브로커 정보 등)를 저장하는 시스템
 - 브로커 상태 변화를 감지하고 공유하는 저장소
 - Controller 브로커가 다른 브로커 상태를 관리할 수 있음
 - ⇒ 클러스터 상태 관리
 - Controller 브로커 선출에 필요한 상태 정보를 제공
 - 여러 대의 서버로 구성된 **ZooKeeper Ensemble(클러스터)** 형태로 운영
- ⇒ Kafka 내부가 아닌, 외부에서 클러스터 상태를 관리하는 시스템

✓ KRaft 모드

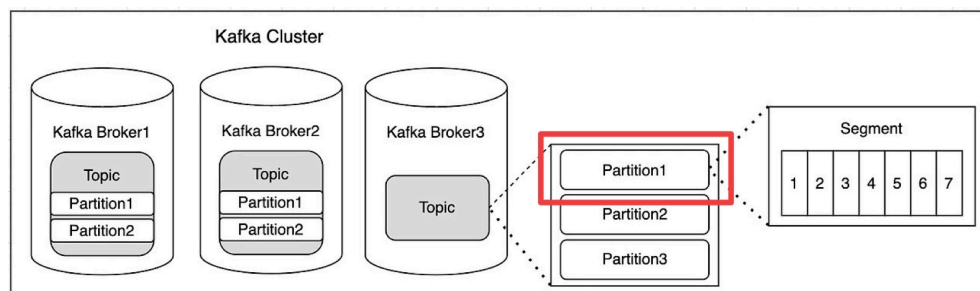
- Kafka는 기존에 ZooKeeper(외부)를 사용했지만, 최근에는 이를 제거하고 KRaft 모드로 전환되고 있는 추세
- 기존에는 외부에서 ZooKeeper가 담당하던 메타데이터 관리와 컨트롤 기능을 Kafka 내부에서 직접 수행하는 구조로 변화

3-3. Topic (토픽)



- 하나의 Broker에는 여러 토픽(Topic)과 파티션(Partition)이 존재하며, 데이터를 분산 저장함
- 브로커에서 데이터를 구분하고 관리하는 단위
 - 쉽게 말해, 어떤 주제의 데이터를 보관할 지 정하는 기준
 - e.g. 사용자 로그, 주문 정보, 클릭 데이터
- 전통적인 메시지 큐와 달리 데이터를 읽은 후에도 삭제되지 않고 로그 형태로 계속 저장됨
- 여러 Consumer가 같은 데이터를 독립적으로 읽을 수 있고, 필요 시 과거 데이터를 다시 읽어 처리할 수 있음

3-4. Partition (파티션)



- 하나의 토픽은 여러 개의 파티션으로 나뉘어 저장됨
- 데이터를 나누어 저장함으로써 '병렬 처리' 가능
 - 파티션 수가 증가하면 처리량은 증가하지만,
 - 장애 발생 시 복구해야 할 데이터도 증가하는 trade-off 존재
- 각 파티션 내부에서는, 데이터가 순서대로 저장되며, 순서는 파티션 내부에서만 보장됨
 - 즉 다른 파티션 간의 순서 보장이 되지 않음.

• 파티션 구조 :

1. 리더 파티션(Leader Partition)

Producer와 Consumer가 직접 통신하는 파티션 ⇒ 그래서 리더라고 함

- 데이터의 읽기/쓰기 요청을 처리

2. 팔로워 파티션(Follower Partition)

Producer가 리더 파티션에 전달한 데이터를 복제(replication)하여 저장함

💡 복제 계수(Replication Factor)

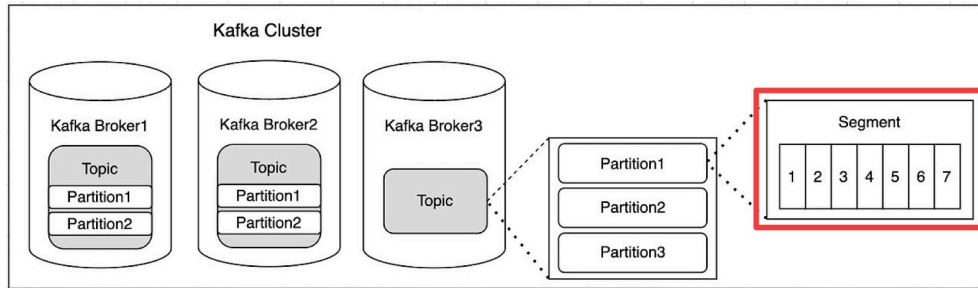
: 하나의 파티션을 몇 개로 복제하여 저장할지 결정하는 값

- 예를 들어, 복제 계수가 1이면 복제 없이 1개만 저장되고, 3이면 동일한 데이터가 3개(Leader 1 + Follower 2)로 저장됨

⇒ 장애 발생 시에도 데이터 유실을 방지할 수 있음

- 리더 파티션이 속해있는 브로커에 장애가 발생한다면, 새로운 리더로 승격될 수 있음

3-5. Segment(세그먼트)



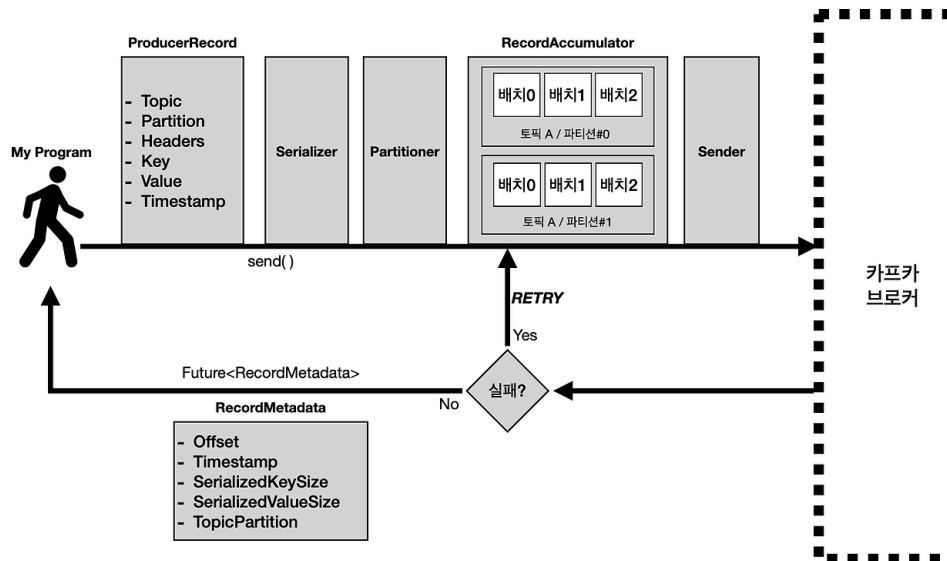
- 파티션 내부에서 데이터를 저장하는 '파일 단위'
- 세그먼트 데이터는 순서대로 기록되며, 일정 기준마다 새로운 파일로 나뉘어 저장됨
→ 각 파티션 내부에서는, 데이터가 순서대로 저장된다

4. Producer

- Kafka 시스템은 Producer → Broker → Consumer 구조로 동작하며, 시작점이 바로 Producer!
- Kafka에서 데이터를 생성하여 브로커로 전송하는 역할
 - Producer는 데이터를 브로커의 리더 파티션으로 전송하고,
 - 이후 해당 데이터는 팔로워 파티션으로 복제되어 저장됨

4-1. Producer 구조와 메시지 전달 과정

: Producer는 데이터를 바로 보내는 것이 아니라, 여러 단계를 거쳐 전송함



• 메시지 전송 흐름 :

ProducerRecord 생성 → send() 호출 → 직렬화 → 파티션 선택 → 배치 → 브로커 전송

1. **ProducerRecord** : Kafka로 전송할 메시지 객체
 - 어디로 무엇을 언제 보낼 지에 대한 정보가 담겨 있음
 - 토픽 정보, **key, value** 등
2. **send()** : Producer에서 메시지 전송을 시작하는 메서드
 - send()를 호출한다고 바로 전송되는 것이 아니라,
 - 직렬화 → 파티션 선택 → 배치 과정을 거친 후, 실제 전송됨
3. **Serializer (직렬화)** : 데이터를 네트워크 전송을 위해 Byte 형태로 변환하는 과정
 - Kafka는 데이터를 그대로 전송하지 않고, 직렬화하여 전송함
4. **partitioner (파티션 선택)** : 직렬화된 **key/value** 정보를 바탕으로, 지정된 토픽 안에서 전송할 파티션 결정
5. **RecordAccumulator (배치)** : 같은 파티션으로 전송될 메시지를 모아두었다가, 한 번에 전송하기 위해 임시로 저장하는 공간
6. **Sender** : 일정 시점이 되면 실제로 Kafka 브로커(지정된 토픽의 리더 파티션)로 데이터 전송

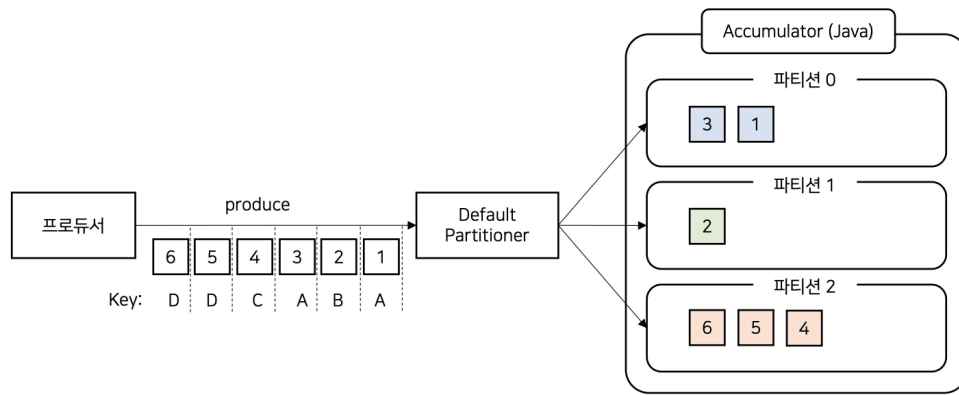
• 전송 결과 :

- a. 성공 → 메타데이터 반환
- b. 실패 → 재시도 → 재시도 실패 시 예외 발생

4-2. Producer partitioner

: Producer 가 생성한 Record를 어떤 Partition 에 저장할지 결정하는 기능

❶ Key가 있는 경우 : key 값을 기반으로 hash 계산

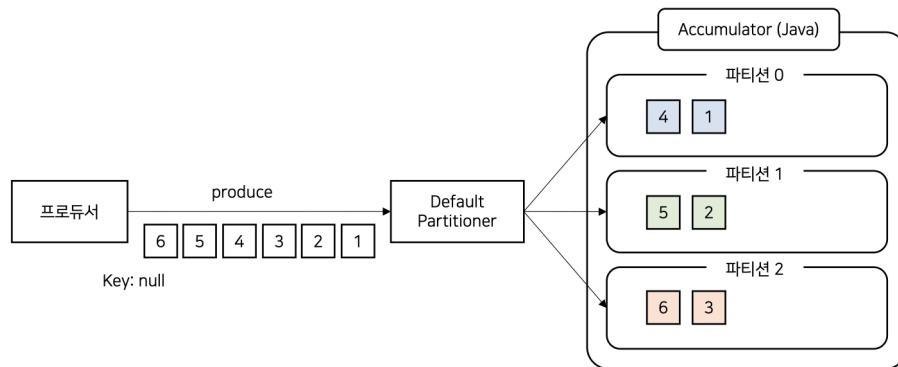


- 동일한 key → 동일한 파티션으로 전송

② Key가 없는 경우 : 파티셔너의 분배 전략에 따라 파티션이 결정

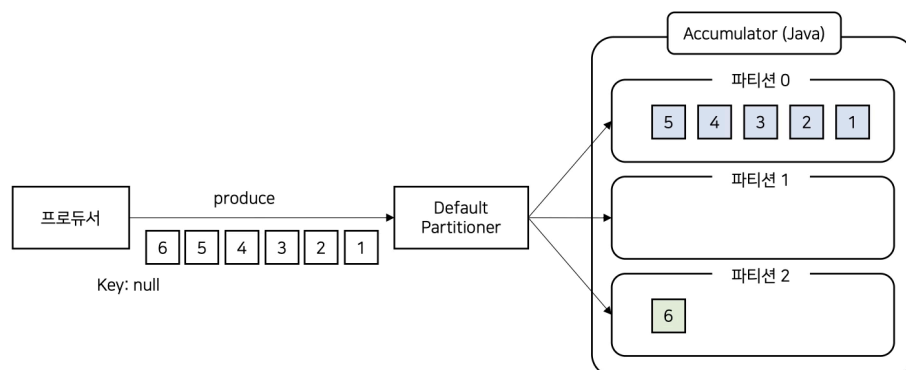
1. Round Robin 방식 (Kafka 2.3 이전 기본 방식)

파티션을 순서대로 돌아가며 메시지를 분배 (균등 분배)



2. Uniform Sticky 방식 (Kafka 2.4 이후 기본)

하나의 파티션에 메시지를 모아서 전송



- batch가 꽉 차거나 전송 시점이 되면 다른 파티션으로 변경
- 배치 효율이 커지고 처리량도 늘어남



Producer의 특징

메시지를 브로커에 전송할 뿐, 누가 데이터를 읽는지 신경 쓰지 않음

⇒ Producer와 Consumer는 서로 독립적으로 동작

⇒ Producer와 Consumer 간 결합이 제거됨 (Decoupling)

⇒ 시스템 확장과 변경이 쉬워짐

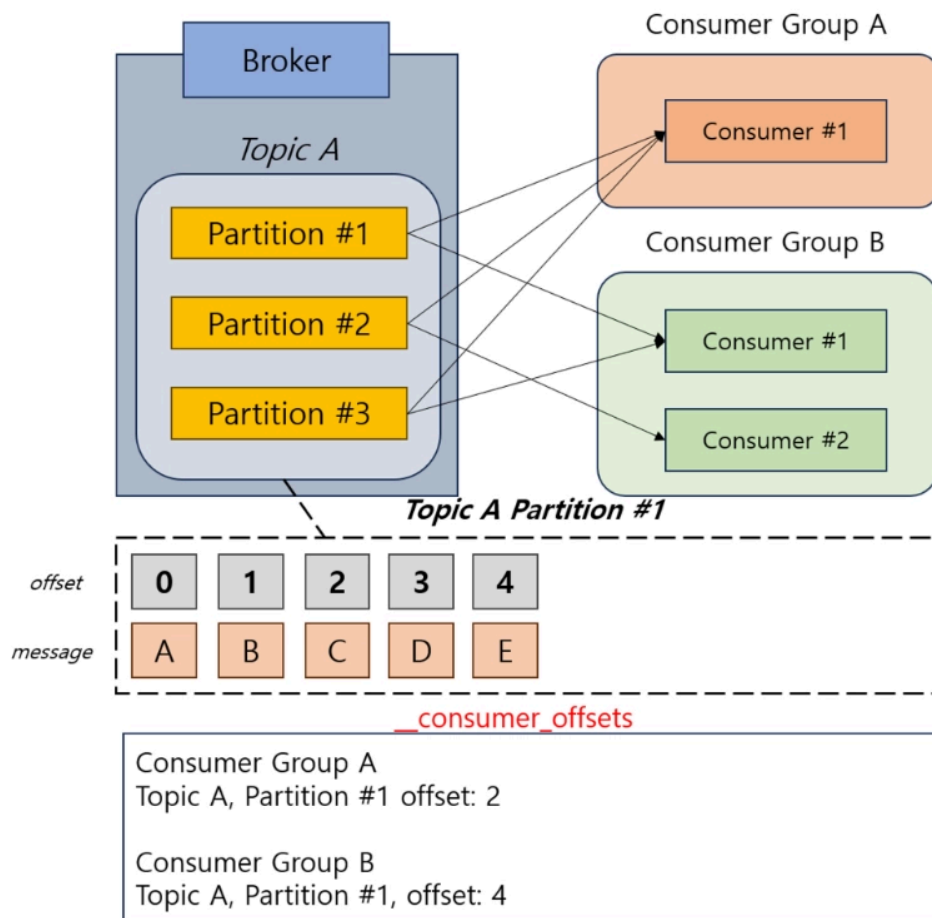
5. Consumer & Consumer Group

5-1. Consumer란?

- 토픽의 파티션에 저장된 메시지를 읽어오는 역할
- 메시지를 어디까지 읽었는지를 나타내는 **offset**을 기준으로 데이터를 소비
- 데이터를 요청할 때, 해당 파티션의 리더가 위치한 브로커와 통신

5-2. Consumer Group이란?

같은 목적을 가진 여러 Consumer를 하나의 그룹으로 묶어, 토픽의 파티션을 나누어 처리하는 구조



- Consumer Group 안의 Consumer A, B, C는 서로 다른 역할을 하는 것이 아니라, **같은 작업을 수행하는 Consumer 인스턴스**
- 각 Consumer는 **자신이 할당받은 파티션들의 메시지만** 처리한다.

```
Partition 0 → Consumer A
Partition 1 → Consumer A
```

Partition 2 → Consumer B
Partition 3 → Consumer C

- 하나의 파티션은 같은 **Consumer Group** 내에서 하나의 **Consumer**만 읽음
- 하나의 **Consumer**는 여러 파티션을 읽을 수 있음 (1:1 또는 n:1 관계)

✓ Consumer Group이 필요한 이유

- Consumer가 1개만 있으면 모든 메시지를 혼자 읽고 처리해야 한다.
- 이때 데이터 유입량이 많아지면 Consumer의 처리 속도가 부족해질 수 있다

⇒ Consumer가 처리하지 못한 메시지가 Kafka에 계속 쌓이게 된다

Kafka로 들어오는 로그: 초당 1,000개
Consumer 1개의 처리량: 초당 300개

✓ Consumer Group을 사용한 '병렬 처리'

Kafka는 이 문제를 해결하기 위해 **Consumer Group** 안의 여러 **Consumer**가 각 파티션을 나누어 병렬로 처리하도록 했음

- 예를 들어 **로그 저장 Consumer** 가 하나만 있으면 모든 로그를 혼자 처리해야 한다.
- 하지만 **로그 저장 Consumer** 를 여러 개 띄우면, 파티션 단위로 작업을 나누어 처리할 수 있다.

같은 역할: 로그 저장

Partition 0 → Log Consumer A
Partition 1 → Log Consumer B
Partition 2 → Log Consumer C

⇒ 각 **Consumer**는 같은 작업을 수행하지만, 서로 다른 파티션을 맡아 처리한다.

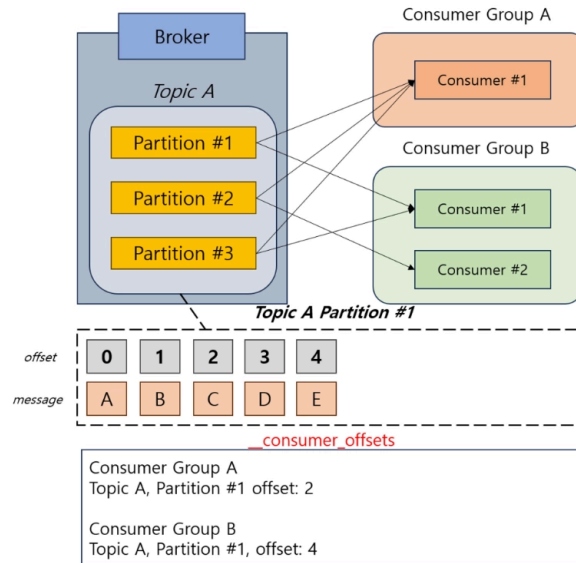
⇒ 이 구조를 통해 전체 처리량을 높일 수 있다

Consumer Group 정리)

- 같은 Consumer Group 안에서는 같은 메시지를 여러 **Consumer**가 중복 처리하지 않는다.
- 하나의 파티션은 같은 Consumer Group 안에서 하나의 Consumer에게만 할당된다.
- 따라서 Consumer Group은 처리량을 높이면서도, 메시지가 중복 소비되지 않도록 파티션 단위로 작업을 분산하는 구조이다.
- Consumer Group의 효과 :
 - 전체 처리량을 높임
 - 특정 Consumer에 장애가 발생해도, 다른 Consumer가 해당 파티션을 이어받아 처리할 수 있음

💡 Offset과 Consumer Group의 개념

- Offset : 각 파티션에서 메시지의 위치를 나타내는 값
- Offset과 Consumer Group 관계
 - offset : Consumer가 어디까지 읽었는지를 기록
 - Kafka에서는 Offset 관리를 Consumer Group 단위로 관리
 - 각 Consumer Group은 독립적인 Offset을 가짐



- `__consumer_offsets` 역할
 - 각 Consumer Group의 Offset 정보를 저장하는 내부 토픽
 - 그룹별 읽기 위치를 관리

5-3. Coordinator (브로커)

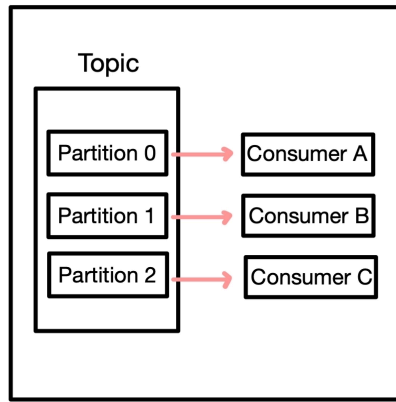
- Consumer Group을 관리하는 브로커
 - kafka 클러스터 안의 여러 브로커 중 하나의 브로커가 '특정 Consumer Group'의 Coordinator 역할을 맡음
 - Consumer Group마다 Coordinator가 하나씩 존재
 - 예를 들어 `log-save-group` 이라는 Consumer Group이 있으면,
Kafka 브로커 중 하나가 이 그룹의 Coordinator가 된다.
- Coordinator(브로커)가 하는 일

Consumer Group 관리
Consumer 참여/이탈 감지
리밸런스 수행
Consumer별 파티션 할당 관리
offset commit 관리

💡 리밸런스(rebalance)

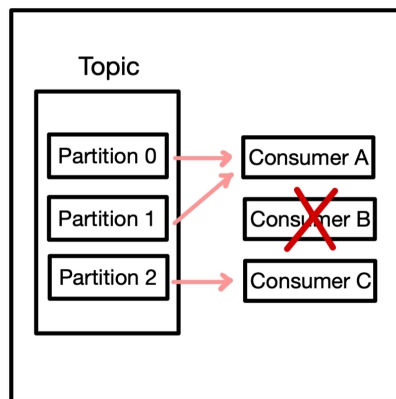
Consumer Group 안에서 Consumer 변화가 생겼을 때, 파티션을 다시 할당하는 과정

- 예를 들어 기존에는 이렇게 처리하고 있었다고 할때,



 변화 발생 : Consumer B가 장애로 빠지면, Partition 1를 처리할 Consumer가 없어진다

- 그러면 Coordinator(브로커)가 Consumer Group 상태를 다시 계산해서 파티션을 재분배 (**Rebalance**)



- Partitions1을 Consumer A 에게 할당

cf) **Coordinator**랑 **Controller**는 다른 개념

둘 다 Kafka 내부에서 "관리 역할"을 하지만, 관리 대상이 다르다

1. Controller → Kafka 클러스터 전체의 브로커/파티션 리더 관리
2. Coordinator → Consumer Group 관리